Disagreement between two recommenders

Counting ranking inversions with merge sort

DAA Assignment 2026 · Batch 12 · OTT / Streaming

23I501 Design and Analysis of Algorithms

Team Members :
24I309 - David Melvin L
24I310 - Dev Prasanna R
24I316 – Gokul D
24I346 – Rakesh M
24I347 – Ramishwar B

Problem modelling

A streaming platform runs two engines that each rank the same n unique titles.

Input and output

Input: a permutation B of 1..n, where B holds the Engine A position of each title in Engine B's order.

Output: the number of pairs (i, j) with i < j and B[i] > B[j].

Objective

Count every disagreement (inversion). This is a counting problem, so there is no optimum to find and no yes/no question to answer.

Given instance

3	1	6	2	8	4	7	5

index 1 to 8, left to right

Expected disagreement score

9

Data structure analysis

Three ways to hold the ranking and count inversions.

Approach	Time	Extra space	Fits when
Array with brute-force pair scan	$O(n^2)$	O(1)	Tiny inputs only
Array with merge-sort counting	O(n log n)	O(n)	A full ranking arrives at once
Fenwick tree (binary indexed tree)	O(n log n)	O(n)	Rankings arrive incrementally

Choice

Merge sort handles the complete nightly ranking in O(n log n). A Fenwick tree supports updates and prefix queries as values arrive, once the values are coordinate-compressed or otherwise indexed.

Design technique: divide and conquer

1 Divide

Split the array into two halves.

2 Recurse

Count the inversions inside each half.

3 Merge

Merge the sorted halves and count the pairs that cross the split.

The counting step

When right[j] < left[i], right[j] is smaller than every remaining element of the sorted left half, so add len(left) - i to the count.

Recurrence

T(n) = 2T(n/2) + O(n), with T(1) = O(1). The halves are independent, and the merge combines their results.

Why the other four techniques do not fit

Technique	Reason
Greedy	A local choice does not count every inverted pair; the merge counting argument is needed.
Dynamic programming	The two halves are independent, so no overlapping subproblems exist and memoization has nothing to reuse.
Backtracking	Checking all pairs costs $O(n^2)$, and enumerating rankings would cost far more. Neither is needed.
Branch and bound	The task counts every inversion instead of searching for one optimal answer, so pruning cannot skip any pair.

> Each half is processed once, so merge sort has no repeated work. Memoization helps when subproblems repeat, as in many DP problems, and would not speed this method up.

Algorithm and pseudocode

```
COUNT(A):
  if length(A) <= 1: return (A, 0)
  split A into L and R
  (L, cL) = COUNT(L)
  (R, cR) = COUNT(R)
  merge L and R into M
    whenever R[j] < L[i],
      add the number of elements left in L
  return (M, cL + cR + cross)
```

Recurrence

$T(n) = 2T(n/2) + O(n)$

$T(1) = O(1)$

Code in the submission

code/main_solution.py: merge-sort counting

code/rejected_alternative.py: brute-force baseline

Trace: recursive splitting

Input [3, 1, 6, 2, 8, 4, 7, 5]



Single elements contain no inversions, so the merging starts from the bottom row.

Trace: every merge

Step	Left half	Right half	Merged	Cross-inversions
1	`[3]`	`[1]`	`[1, 3]`	1
2	`[6]`	`[2]`	`[2, 6]`	1
3	`[1, 3]`	`[2, 6]`	`[1, 2, 3, 6]`	1
4	`[8]`	`[4]`	`[4, 8]`	1
5	`[7]`	`[5]`	`[5, 7]`	1
6	`[4, 8]`	`[5, 7]`	`[4, 5, 7, 8]`	2
7	`[1, 2, 3, 6]`	`[4, 5, 7, 8]`	`[1, 2, 3, 4, 5, 6, 7, 8]`	2

Left subtotal (steps 1 to 3) = 3. Right subtotal (steps 4 to 6) = 4. Final merge adds 2. Total = 3 + 4 + 2 = 9

Proof of correctness

Base case

An array of size 0 or 1 has no inversions, and COUNT returns 0.

Inductive step

Assume both recursive calls count the inversions inside their halves correctly. During a merge, when the next right value is smaller than the current left value, it is also smaller than every remaining value in the sorted left half. Adding the number of remaining left values therefore counts exactly the cross-inversions that this right value contributes.

Every inversion falls in exactly one of three groups

Inside the left half

counted by the left call

Inside the right half

counted by the right call

Across the split

counted during the merge

Complexity and worst case

Merge-sort counting

Time: O(n log n)

Extra space: O(n)

Best, average and worst case are all O(n log n).

Brute force

Time: $O(n^2)$

Extra space: O(1)

Worst-case input

Reverse-sorted [n, n-1, ..., 1]

n(n-1)/2 inversions

n = 8 gives 28

Merge sort runs the same divide-and-merge recursion on every input order, so its growth stays O(n log n) even on the reverse-sorted array, which has the most inversions. The measured constant factor does vary with input order (see the benchmark slide).

Nightly scale estimate

5×10^7 users, up to 10^5 titles per user.

Brute force

2.499975×10^{17}

pair checks across all users

Per user: $n(n-1)/2 = 4{,}999{,}950{,}000$

Merge sort

8.3×10^{13}

scale units across all users

Per user: $n \log_2 n \approx 1.66 \times 10^6$

These are theoretical operation counts, not wall-clock runtimes. The gap is about 3,000 times in operations.

Changed requirement: weighted top-10 disagreements

Assumption

A title is top-10 if it appears in Engine A's top 10 or Engine B's top 10. The brief does not say which engine defines top-10, so confirm this reading with the instructor.

Weights

An inversion involving at least one top-10 title weighs 5. Every other inversion weighs 1.

Optimized method

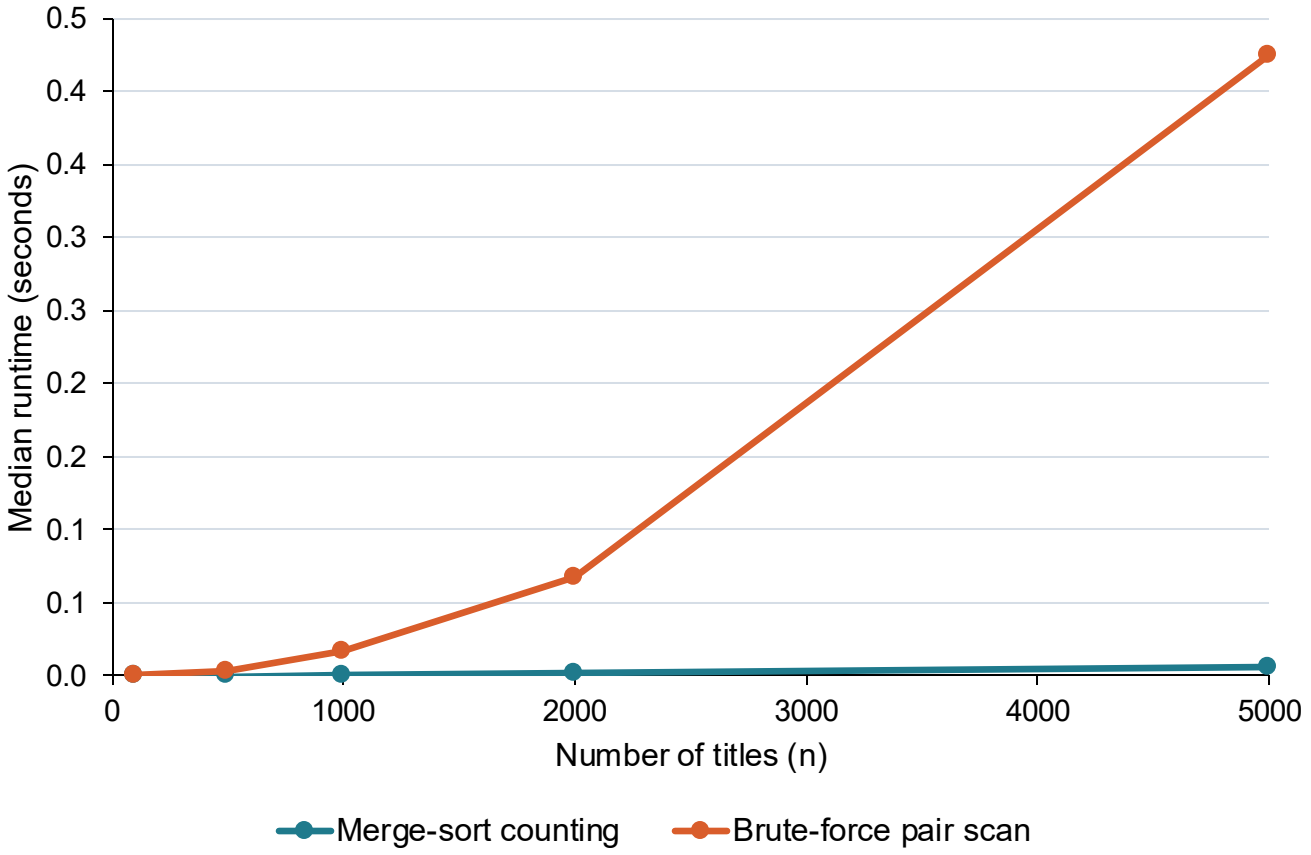
Keep a suffix count of top-10 titles in the left half during each merge.

For a right element that forms cross-inversions, add the number of cross-inversions plus 4 extra units per qualifying pair.

Time stays O(n log n) and extra space stays O(n). A brute-force weighted function in the code validates the result.

In the 8-title example every title is top-10, so the weighted score is 9 × 5 = 45.

Implementation and empirical analysis

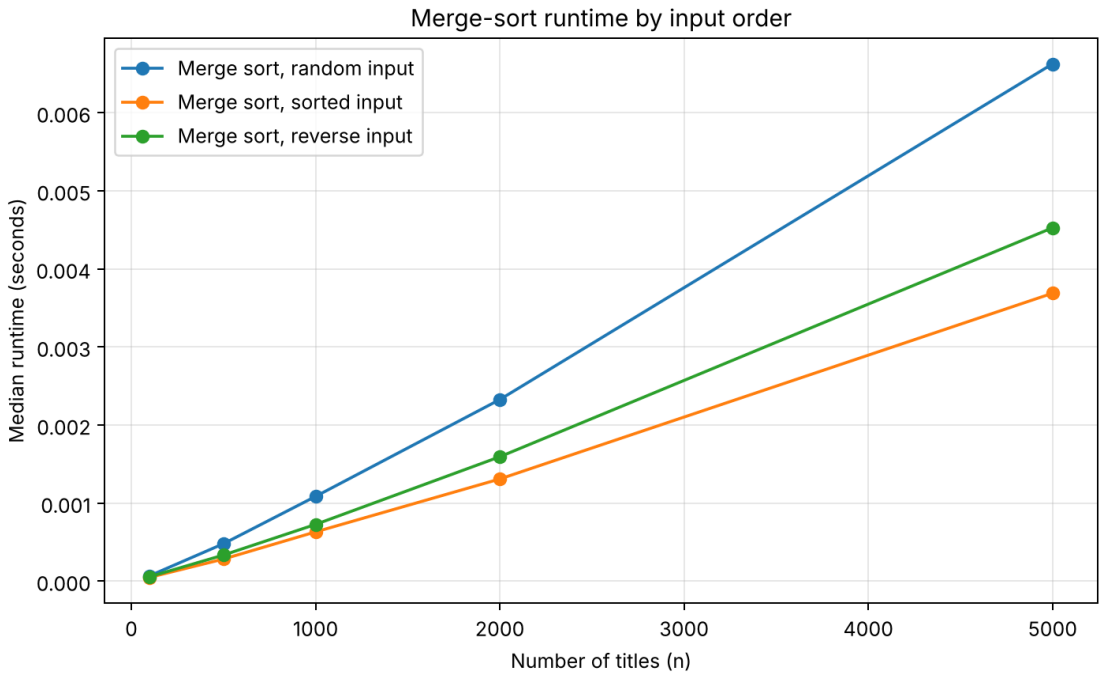


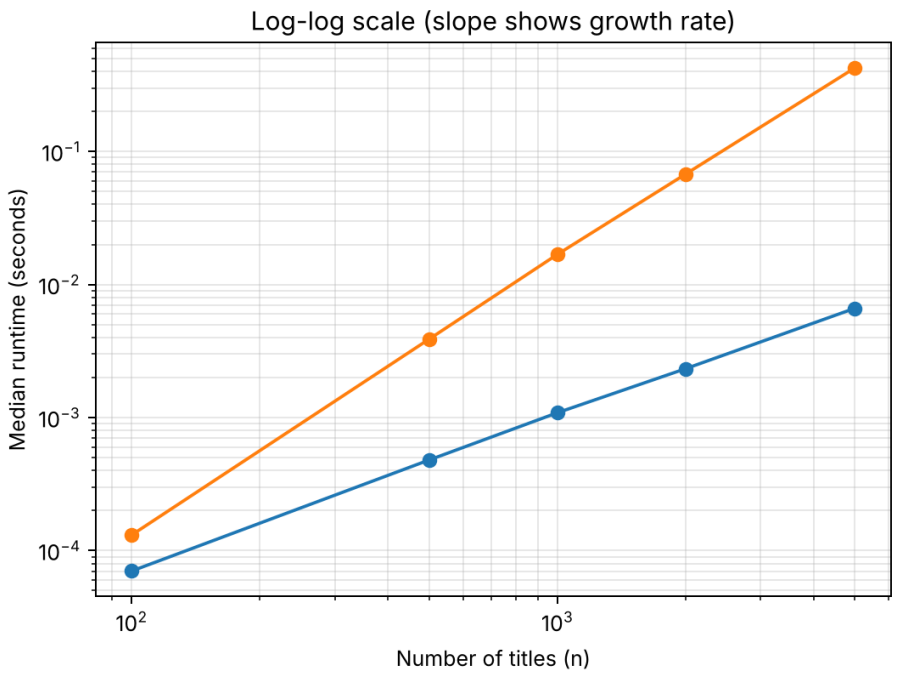
n	Merge sort (ms)	Brute force (ms)
100	0.07	0.13
500	0.48	3.89
1,000	1.09	16.84
2,000	2.33	67.57
5,000	6.63	425.29

At n = 5,000 the brute-force scan takes about 64 times as long as merge sort.

Median of five trials on the same deterministic random permutations. Both algorithms return the same inversion count before any timing is recorded. Runtimes vary with the computer and Python version. Files: results/timing.csv, timing_plot.png and timing_orders.csv.

Growth rate and input order





Input order

At n = 5,000 merge sort took 3.7 ms on sorted, 4.5 ms on reverse-sorted and 6.6 ms on random input. The curves differ by a constant factor and share one growth rate.

Log-log view

On log-log axes a power law is a straight line. Brute force climbs steeply, while merge sort rises slowly and stays below it at every size.

Conclusion and reference

Conclusion

Merge-sort inversion counting measures ranking disagreement in $O(n \log n)$, against $O(n^2)$ for brute force. That suits a nightly batch over complete rankings. Fenwick trees are the better fit when rankings arrive incrementally.

Reference

Donald E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd edition, Addison-Wesley, 1998.

A general reference on sorting and permutations. It does not describe any named OTT platform's recommender pipeline.